E5FR
Année 2025 - 2026

Rapport de projet Plateforme de Gestion de Contenus Sécurisée

Groupe: Yasin GUNDOGDU (yasin.gundogdu@edu.esiee.fr)
& Mike CUNHA (mike.cunha@edu.esiee.fr)

Sommaire

Introduction : L'Idée "SafeBoard" - Portail d'intégration RH sécurisé **3**
Phase 1 - A : Configuration et gestion des buckets AWS S3 **4**
Objectif : Établir une infrastructure de stockage sécurisée pour les contenus numériques. 4
1. Création des compartiments S3 4
2. Politiques restrictives, versioning et journalisation 4
3. Gestion des inventaires 7
Phase 1-B : Architecture et gestion des coûts **8**
Objectif : Concevoir une architecture cloud scalable et gérer les coûts associés. 8
1. Diagramme architectural 8
2. Estimation des coûts 8
Phase 2 : Mise en œuvre de la sécurité dans les environnements AWS **10**
Objectif : renforcer la sécurité réseau et des accès. 10
1. Configuration du Virtual Private Cloud (VPC) 10
2. Sécurisation par Groupes de Sécurité (SG) et ACL 11
Phase 3 : Gestion des clés et journalisation dans AWS **13**
Objectif : Mettre en place une gestion sécurisée des clés de chiffrement et la journalisation des activités. 13
1. Chiffrement avec AWS KMS 13
2. Gestion des secrets applicatifs 16
Phase 4 : Surveillance et conformité dans AWS **18**
Objectif : Mettre en place une surveillance continue et assurer la conformité aux meilleures pratiques. 18
1. Audit et Surveillance Continue 18
2. Maintien de la Conformité 20
Phase 5 : Application de DevSecOps **23**
Objectif : Intégrer la sécurité dans le cycle de vie du développement logiciel. 23
1. Configuration de l'environnement de build 23
2. Test du Pipeline et Résolution (La boucle DevSecOps) 26
Recherche Théorique : Les attaques adverses ciblant les plateformes de contenus numériques **28**
Objectif : Répondre à la consigne de recherche sur l'état de l'art du cahier des charges 28
Conclusion **30**

Introduction : L'Idée "SafeBoard" - Portail d'intégration RH sécurisé

Dans le cadre de ce projet de Sécurité Cloud, nous avons choisi de concevoir et de déployer **SafeBoard**. Il s'agit d'une plateforme web interne destinée à faciliter l'intégration des nouveaux employés d'une entreprise.

La gestion des ressources humaines implique le traitement de données à caractère personnel (PII) hautement critiques. Sur cette plateforme, le nouvel employé est amené à :

1. **Uploader des documents sensibles :** Pièces justificatives d'identité (carte d'identité, passeport), relevés d'identité bancaire (RIB), et contrats de travail signés.
2. **Consulter des contenus numériques :** Visionner des vidéos de formation à la cybersécurité interne ou lire des manuels d'intégration.

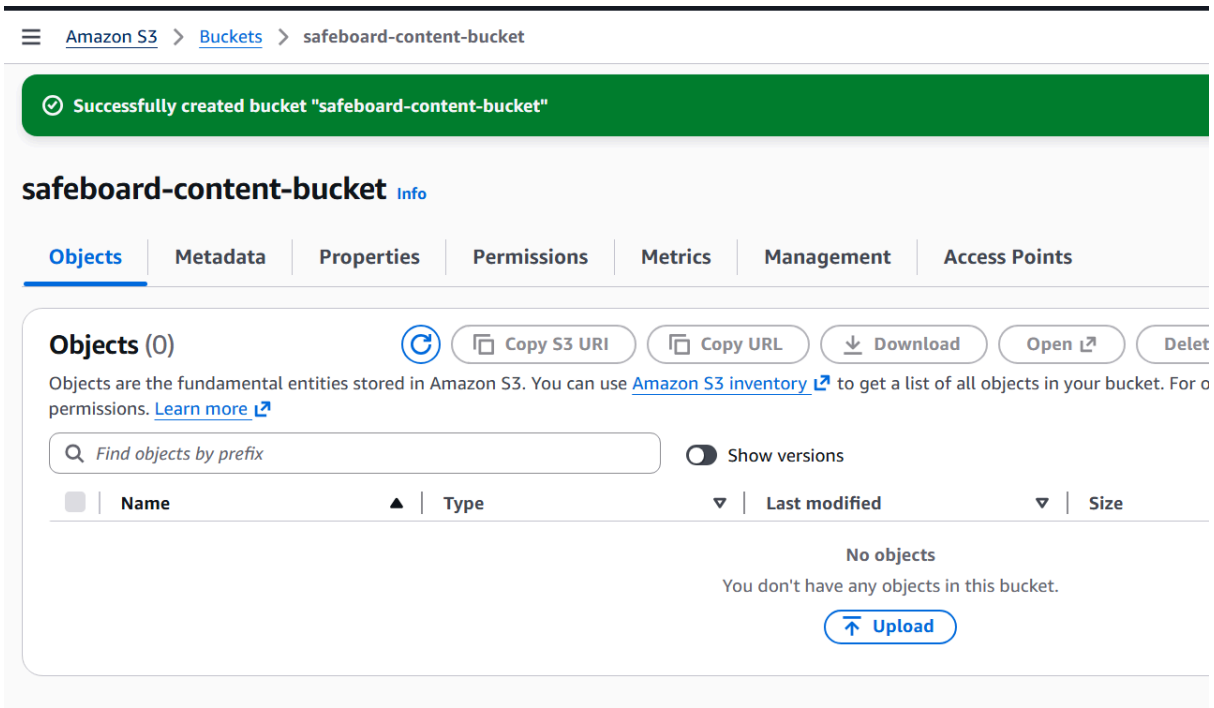
La nature extrêmement confidentielle de ces documents (données bancaires, identités) justifie pleinement le déploiement d'une infrastructure cloud répondant aux standards de sécurité les plus stricts : chiffrement de bout en bout, isolation réseau, surveillance continue et intégration de pratiques DevSecOps.

Phase 1 - A : Configuration et gestion des buckets AWS S3

Objectif : Établir une infrastructure de stockage sécurisée pour les contenus numériques.

1. Création des compartiments S3

Nous avons débuté par la création d'un bucket de stockage principal nommé safeboard-content-bucket. Ce compartiment a pour vocation de stocker les fichiers de contenus (articles, images, vidéos de formation) ainsi que les documents RH uploadés par les utilisateurs.



2. Politiques restrictives, versioning et journalisation

La sécurité des données au repos commence par un contrôle d'accès strict. Nous avons appliqué le paramètre **"Block all public access"** (Bloquer tout l'accès public) sur ce bucket afin d'empêcher toute fuite de données accidentelle sur Internet. De plus, le **versioning a été activé**. Cette fonctionnalité est cruciale : elle permet de conserver les différentes variantes d'un objet et d'offrir une protection contre les suppressions ou les écrasements accidentels des documents RH.

Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policie
access. These settings apply only to this bucket and its access points. AWS recommends that you
access. If you require some level of public access to your buckets or objects within, you can custo

Block *all* public access

On

▶ **Individual Block Public Access settings for this bucket**

Amazon S3 > Buckets > safeboard-content-bucket

Bucket overview

AWS Region

Europe (Ireland) eu-west-1

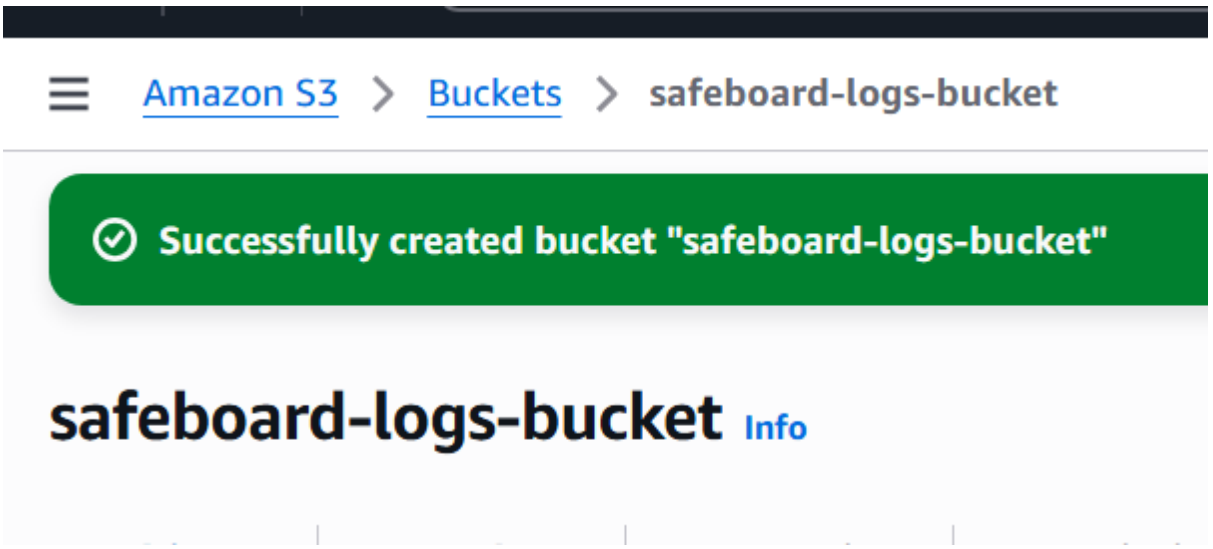
Bucket Versioning

Versioning is a means of keeping multiple variants of an object i
With versioning, you can easily recover from both unintended us

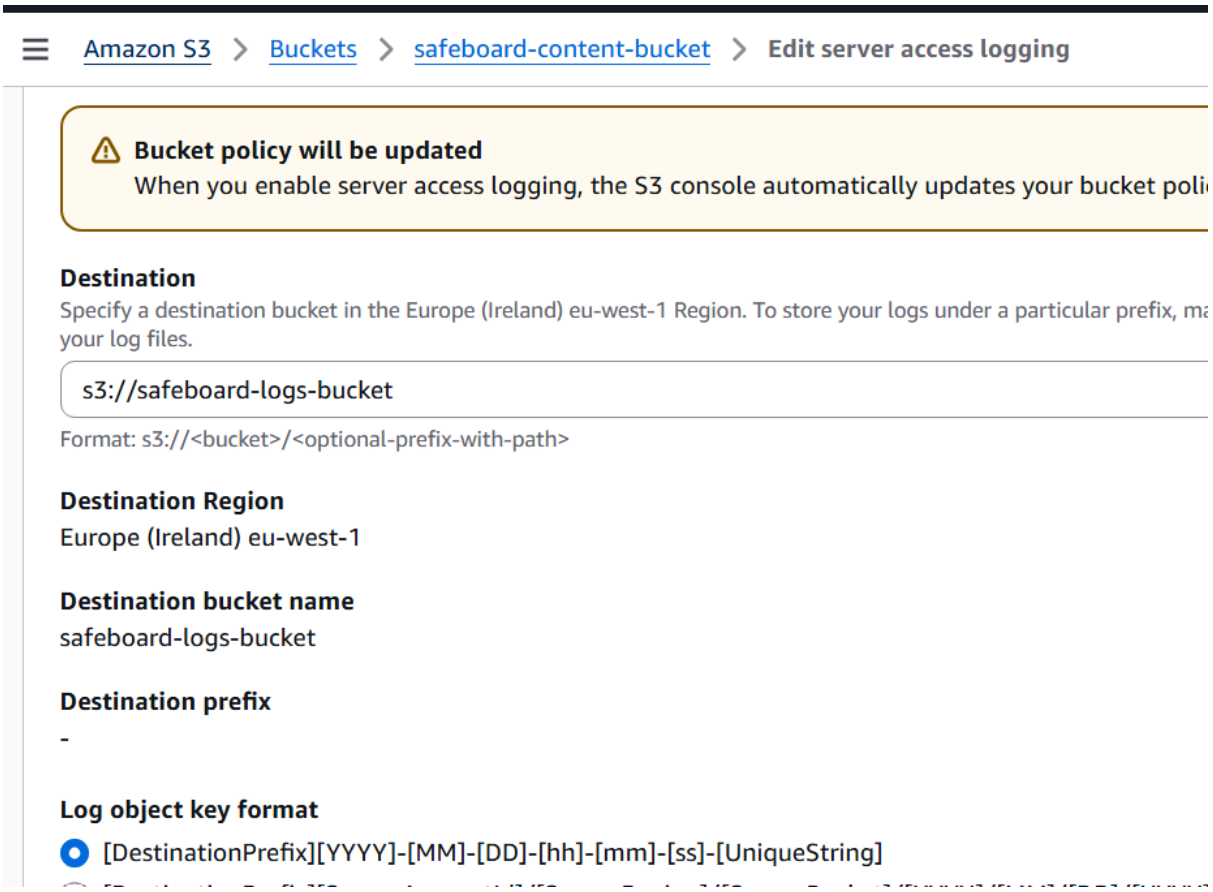
Bucket Versioning

Enabled

Pour assurer la traçabilité complète des accès, nous avons créé un second bucket nommé safeboard-logs-bucket. Ce bucket est également privé, mais ne nécessite pas de versioning. Nous avons ensuite activé la **journalisation des accès au serveur (Server access logging)** sur le bucket de contenu, en désignant le bucket de logs comme destination. Ainsi, chaque requête effectuée vers nos documents est tracée et auditable.



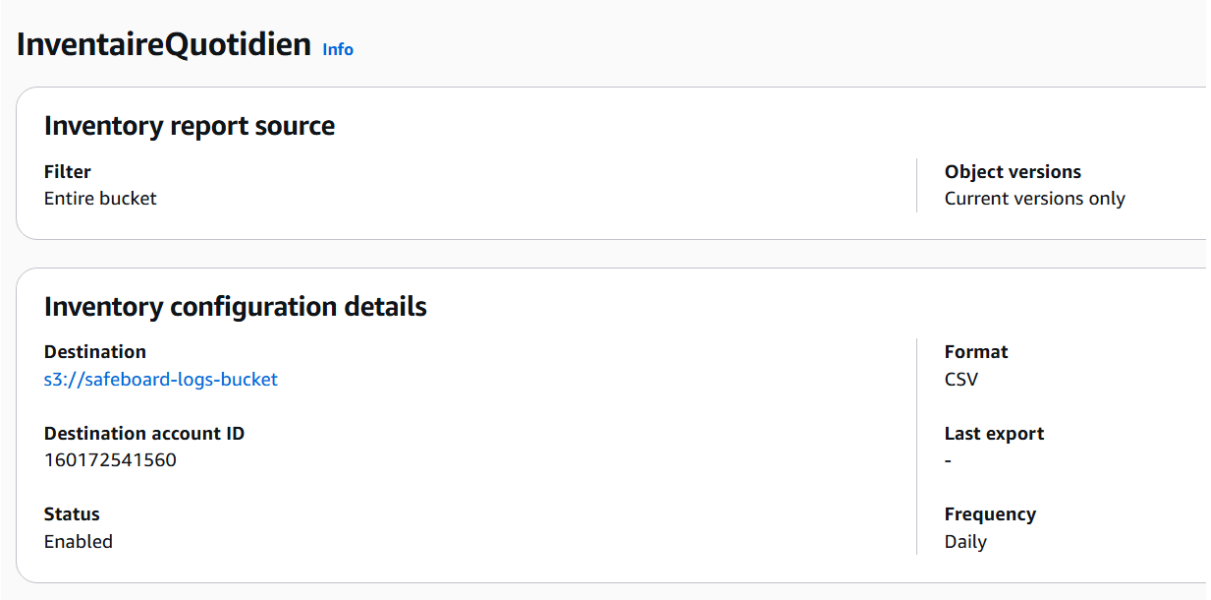
On active la journalisation comme ceci :



Enfin, nous avons implémenté une "Bucket Policy" restrictive qui impose le chiffrement des données en transit. Cette politique interdit (Deny) explicitement toute action sur le bucket si la requête ne provient pas d'une connexion chiffrée (HTTPS).

3. Gestion des inventaires

Pour maintenir une visibilité complète sur les actifs stockés, nous avons configuré un rapport d'inventaire nommé InventaireQuotidien. Tous les jours, AWS génère automatiquement un rapport listant l'intégralité des objets du bucket de contenu et le dépose au format CSV dans le bucket de logs. Cette fonctionnalité est idéale pour l'analyse de données de sécurité.

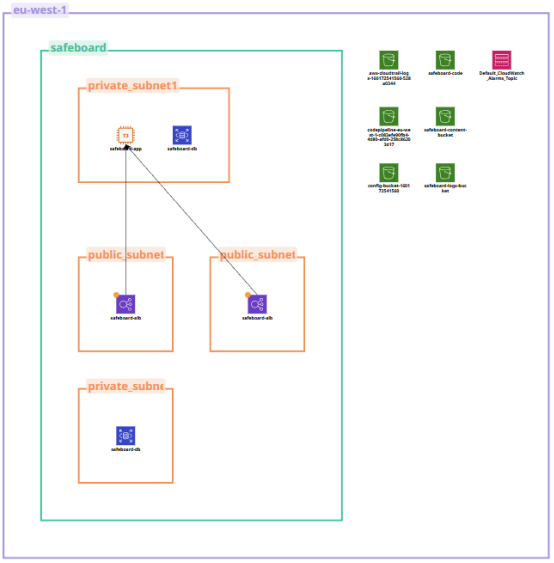


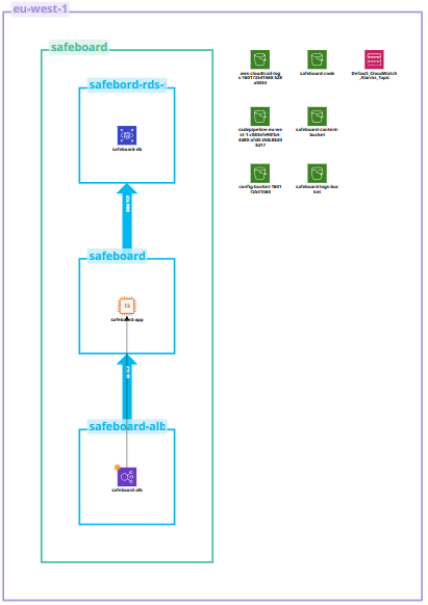
Phase 1-B : Architecture et gestion des coûts

Objectif : Concevoir une architecture cloud scalable et gérer les coûts associés.

1. Diagramme architectural

Pour répondre aux exigences du projet, nous avons conceptualisé notre infrastructure à l'aide de l'outil Cloudcraft. L'architecture s'articule autour d'un VPC (Virtual Private Cloud) isolé. La zone publique héberge notre Application Load Balancer (ALB), qui sert de point d'entrée unique. Nos serveurs d'application (instances EC2) et notre base de données (RDS) sont relégués dans des sous-réseaux privés, inaccessibles directement depuis Internet, illustrant le principe de défense en profondeur.





2. Estimation des coûts

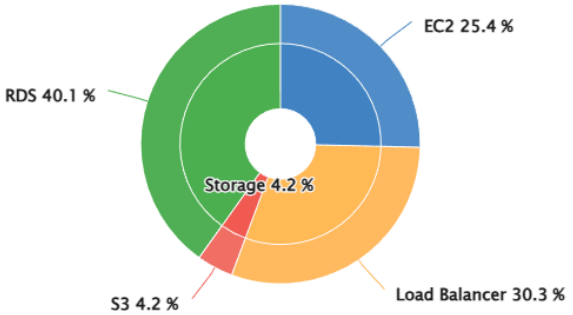
Nous avons utilisé l'AWS Pricing Calculator pour estimer le coût mensuel de notre infrastructure en région Irlande (eu-west-1). Le budget de développement est estimé à **32,77$ / mois**. La répartition des coûts est la suivante :

- **Base de données RDS (MySQL t3.micro) :** ~40,1% du budget (13,14 $/mois).
- **Mise en réseau (Application Load Balancer) :** ~30,3% du budget (9,93 $/mois).
- **Calcul (EC2 t3.micro) :** ~25,4% du budget (8,32 $/mois).
- **Stockage (S3) :** ~4,2% du budget (1,38 $/mois).

Budget for Development eu-west-1

MONTHLY | EFFECTIVE RATE | USD | EXPORT

Compute	$8.32 / mo
Networking	$9.93 / mo
Storage	$1.38 / mo
Database	$13.14 / mo
App services & Integrations	$0.00 / mo
Total	$32.77 / mo



TYPE | REGION | COUNT | PRICE

EC2
Region: EU-WEST-1 | Count: 1
Platform: Linux | Instance type: T3 - Burst/Intel | Size: micro | Billing option: On-Demand 100% | $8.32 / mo
3.1 GHz Intel Skylake E5 2686 v5 | 2 vCPU | 1 GiB memory | EBS only | Up to 2085 Mbps EBS throughput | Up to 5 Gigabit networking performance

Load Balancer
Region: EU-WEST-1 | Count: 1
Type: Application | LCUs: 1 | $9.93 / mo
25 New conn. / sec. | 3000 Active conn. / min. | 1 GB / hour

S3
Region: EU-WEST-1 | Count: 6
Volume Type: Standard | Storage (GB): 10 | PUT/COPY/POST / mo (K): 0 | GET and other / mo (K): 0 | $1.38 / mo

RDS
Region: EU-WEST-1 | Count: 1
Role: Primary | Engine: MySQL | Instance type: T3 - Burst/Intel | Size: micro | Deployment option: Single-AZ | Billing option: On-Demand 100% | $13.14 / mo
Intel Skylake E5 2686 v5 (2.5 GHz) | 2 vCPU | 1 GiB memory | EBS Only | Low to Moderate networking performance

SNS Topic
Region: EU-WEST-1 | Count: 1
Requests / month (K): 1 | $0.00 / mo

Phase 2 : Mise en œuvre de la sécurité dans les environnements AWS

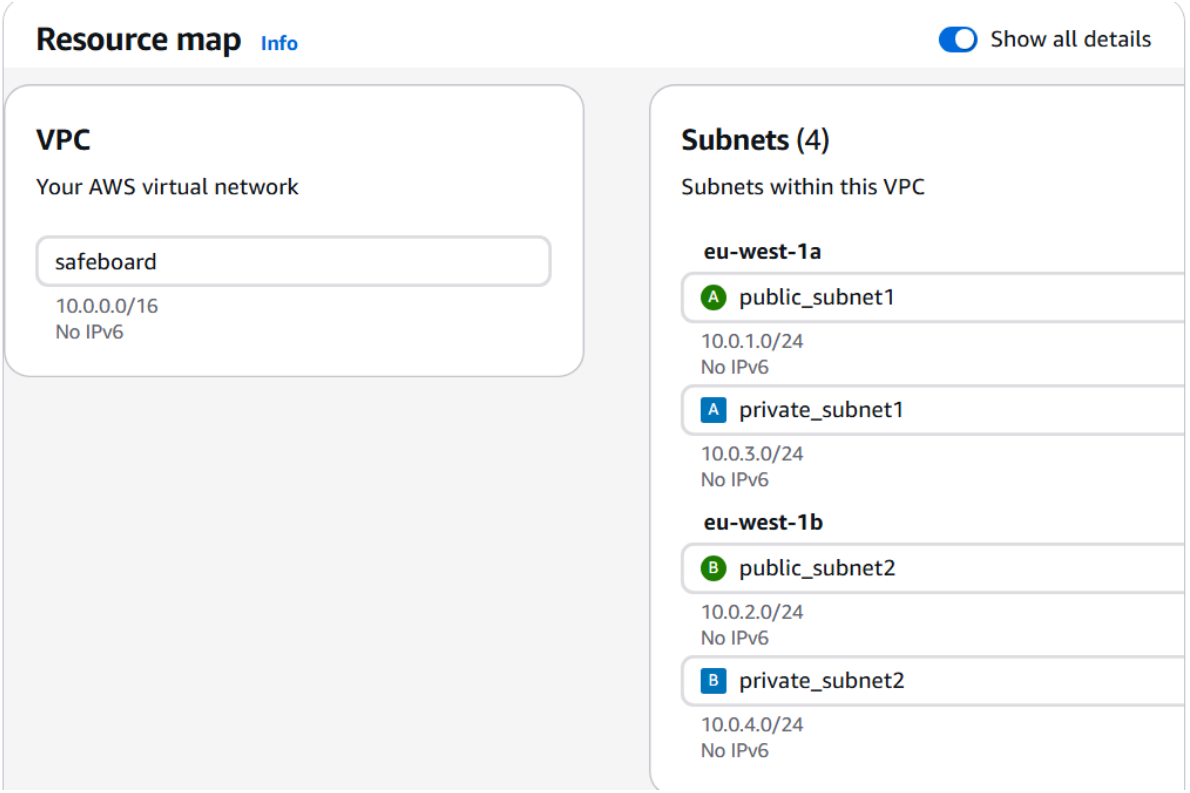
Objectif : renforcer la sécurité réseau et des accès.

1. Configuration du Virtual Private Cloud (VPC)

Nous avons déployé un VPC nommé safeboard avec un bloc d'adresses CIDR de 10.0.0.0/16. Pour assurer une haute disponibilité et une stricte ségrégation réseau, il a été découpé en quatre sous-réseaux répartis sur deux zones de disponibilité (eu-west-1a et eu-west-1b) : deux sous-réseaux publics (10.0.1.0/24, 10.0.2.0/24) et deux sous-réseaux privés (10.0.3.0/24, 10.0.4.0/24).

Les tables de routage reflètent cette sécurité : les sous-réseaux publics ont une route vers l'Internet Gateway (igw-*) , tandis que les sous-réseaux privés n'accèdent à Internet que pour des mises à jour sortantes via une passerelle NAT (nat-*).

Afin de sécuriser les flux vers le stockage, nous avons provisionné un **VPC Endpoint S3 (de type Gateway)**. Cela permet à nos instances privées de communiquer avec nos compartiments S3 sans jamais faire transiter les données sensibles par l'Internet public.



Nos sous-réseaux:

Subnets (4) Info

Find subnets by attribute or tag

Name	Subnet ID	State
public_subnet1	subnet-0d0438e0798d78ca9	Available
public_subnet2	subnet-0d0bb6a72957dff6c	Available
private_subnet1	subnet-0219436fc79cbc403	Available
private_subnet2	subnet-035cfc98046b3b8d0	Available

2. Sécurisation par Groupes de Sécurité (SG) et ACL

La protection réseau opère à deux niveaux complémentaires :

<u>Au niveau des sous-réseaux (Network ACLs) :</u>

Contrairement aux groupes de sécurité, les NACL sont *stateless* (sans état). Nous avons donc créé deux NACL distinctes :

- NACL Publique : Autorise le trafic entrant HTTP/HTTPS (ports 80/443) et configure spécifiquement l'ouverture des ports éphémères TCP (1024-65535) en entrée comme en sortie pour permettre à l'ALB de gérer le trafic de retour.
- NACL Privée : Protège nos serveurs et bases de données en n'autorisant en entrée que le trafic HTTP provenant du CIDR du VPC, et le port MySQL.

Edit inbound rules Info

Inbound rules control the incoming traffic that's allowed to reach the VPC.

Rule number Info	Type Info	Protocol Info	Port range Info	Source Info	Allow/Deny Info	
100	HTTP (80)	TCP (6)	80	0.0.0.0/0	Allow	Remove
110	HTTPS (443)	TCP (6)	443	0.0.0.0/0	Allow	Remove
120	Custom TCP	TCP (6)	1024-65535	0.0.0.0/0	Allow	Remove
*	All traffic	All	All	0.0.0.0/0	Deny	

Add new rule | Sort by rule number

Edit outbound rules Info

Outbound rules control the outgoing traffic that's allowed to leave the VPC.

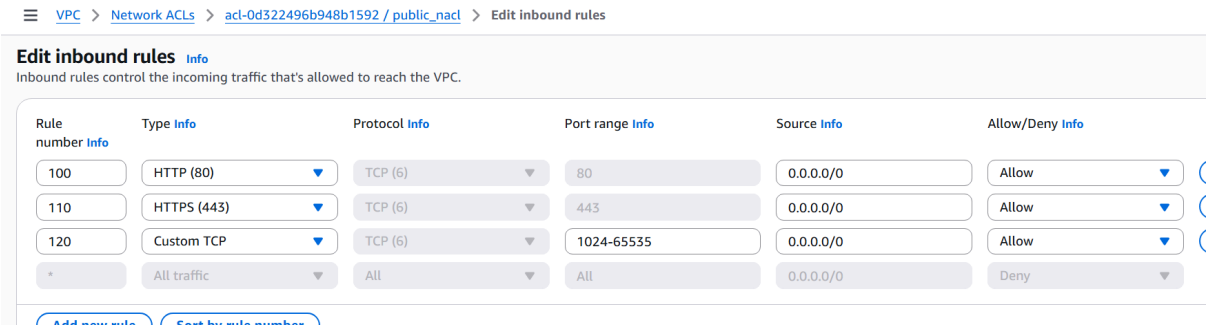
Rule number Info	Type Info	Protocol Info	Port range Info	Destination Info	Allow/Deny Info	
100	HTTP (80)	TCP (6)	80	0.0.0.0/0	Allow	Remove
110	HTTPS (443)	TCP (6)	443	0.0.0.0/0	Allow	Remove
120	Custom TCP	TCP (6)	1024 - 65535	0.0.0.0/0	Allow	Remove
*	All traffic	All	All	0.0.0.0/0	Deny	

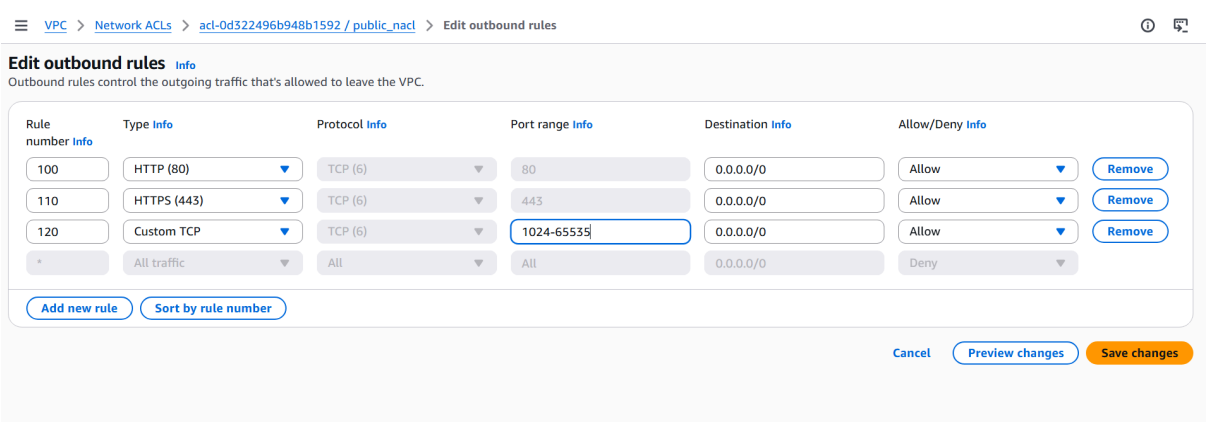
Add new rule | Sort by rule number

Cancel | Preview changes | Save changes

Au niveau des instances (Security Groups) : Les groupes de sécurité appliquent le principe du moindre privilège:

- safeboard-alb-sg : Autorise le trafic HTTP (80) depuis n'importe où (0.0.0.0/0).
- safeboard-ec2-sg : Le serveur web n'accepte le port 80 que depuis le SG du Load Balancer.
- safeboard-rds-sg : La base de données MySQL n'autorise le port 3306 que si la requête provient spécifiquement du SG de l'instance EC2. Aucune connexion sortante n'est autorisée pour la base, garantissant une sécurité maximale.





Phase 3 : Gestion des clés et journalisation dans AWS

Objectif : Mettre en place une gestion sécurisée des clés de chiffrement et la journalisation des activités.

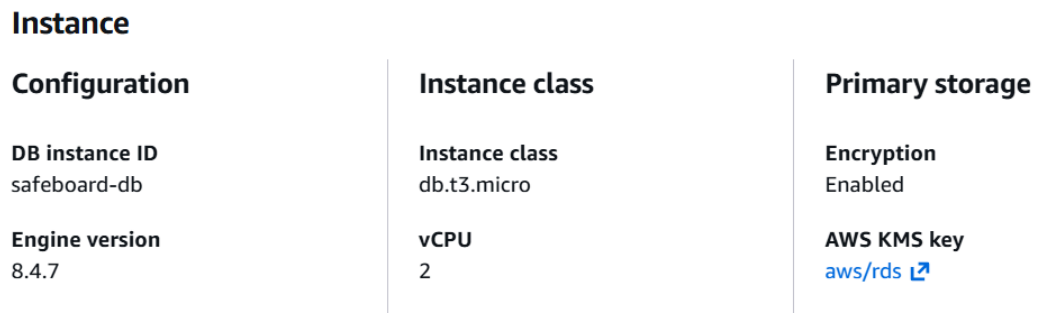
1. Chiffrement avec AWS KMS

Pour nous conformer aux exigences de sécurité, nous avons remplacé le chiffrement par défaut par une clé gérée par le client (Customer Managed Key - CMK). Nous avons créé la clé symétrique safeboard-kms-key. Cette clé offre un contrôle total sur l'audit de son utilisation via CloudTrail.

Nous avons appliqué cette clé KMS au bucket safeboard-content-bucket via l'option "Server-side encryption with AWS KMS keys (SSE-KMS)", garantissant que tous les documents RH (les contrats, pièces d'identité) sont chiffrés au repos.

Pour que notre serveur web puisse lire son propre disque dur (volume EBS) chiffré par cette clé, nous avons ajouté le rôle IAM de l'instance EC2 (system-manager-role) aux utilisateurs autorisés de la clé KMS. Le chiffrement du volume EBS existant s'est opéré en créant une image (AMI) de notre instance, puis en copiant cette AMI en activant l'option "Encrypt EBS snapshots of AMI copy" avec notre clé safeboard-kms-key. La nouvelle instance sécurisée a été lancée à partir de cette AMI chiffrée. De même, le chiffrement KMS a été activé lors de la création de la base de données RDS.

Notre base de donnée utilise KMS :



Tâches :

1. Utiliser AWS Key Management Service (KMS) pour créer et gérer les clés de chiffrement
On crée la clé pour chiffrer le volume EBS et S3.

Customer managed keys (1)

Key actions | Create key

Filter keys by properties or tags

Aliases	Key ID	Status	Key type	Key spec	Key usage
safeboard-kms-key	011df5eb-b47f-4f...	Enabled	Symmetric	SYMMETRIC_DEFA...	Encrypt and decrypt

L'administrateur est le compte admin :

Key administrators (2)

Choose the IAM users and roles who can administer this key through the KMS API. You might need to add additional permissions administer this key from this console. Learn more

Search Key administrators

Name	Path	Type
Admin	/	User

On ajoute au rôle de la machine ec2 actuelle les droits d'utiliser la clé KMS pour qu'elle puisse déchiffrer son volume EBS:

Key users (3)

Add | Remove

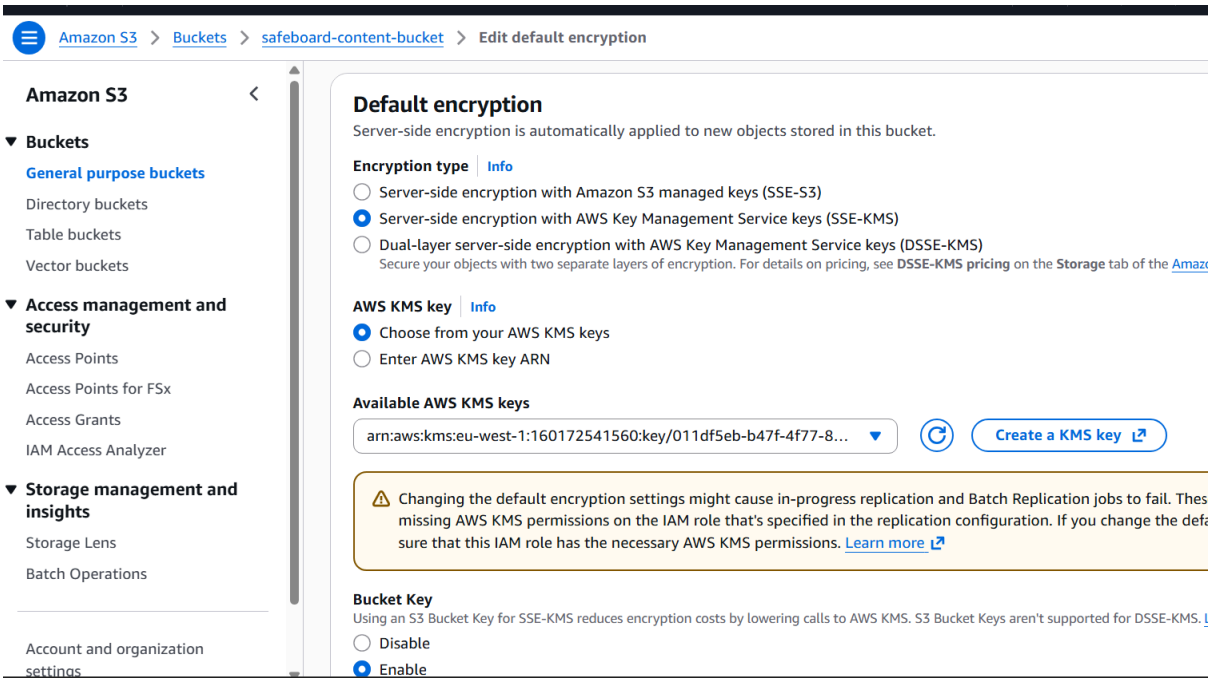
The following IAM users and roles can use this key for cryptographic operations. They can also allow AWS services that are integrated with KMS to use the key on their behalf. Learn more

Search Key users

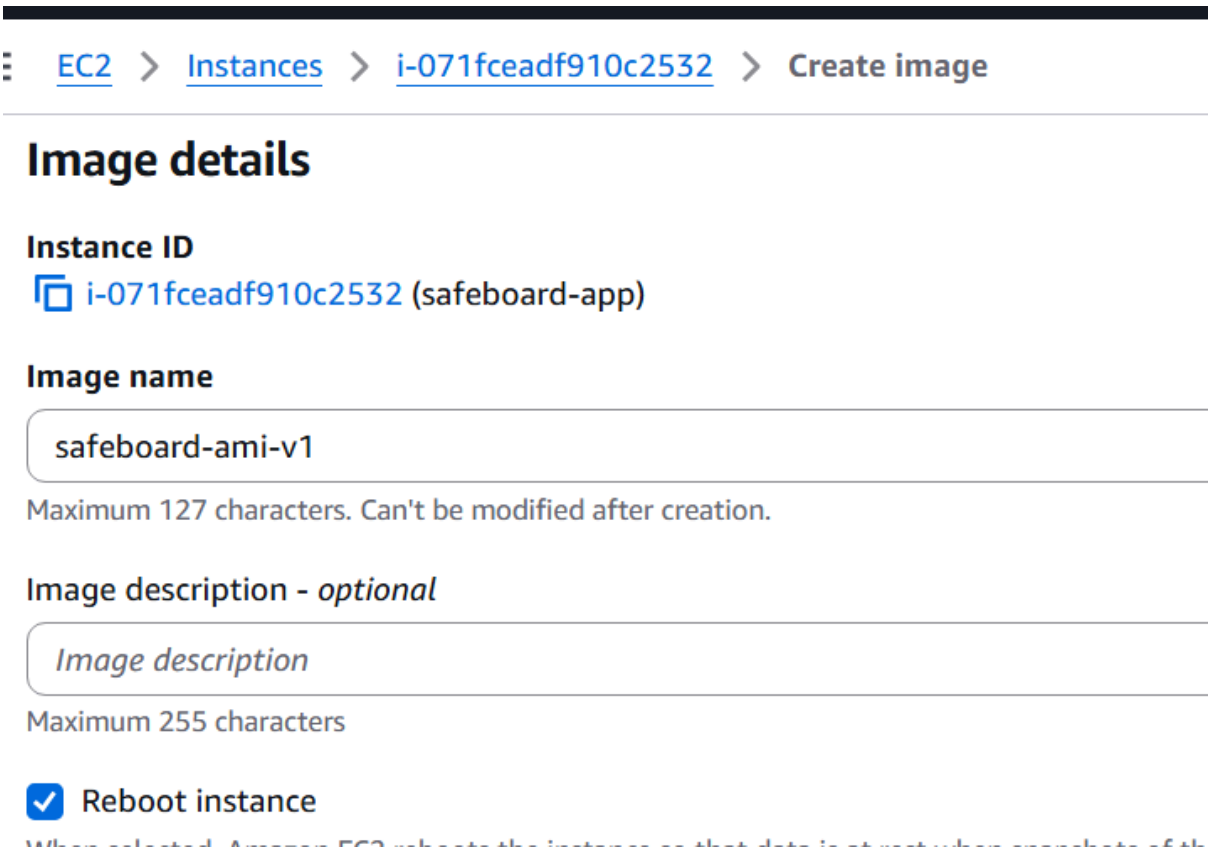
Name	Path	Type
system-manager-role	/	Role
Admin	/	User

2. Chiffrer les données sensibles stockées dans S3 et les volumes EBS des instances EC2.

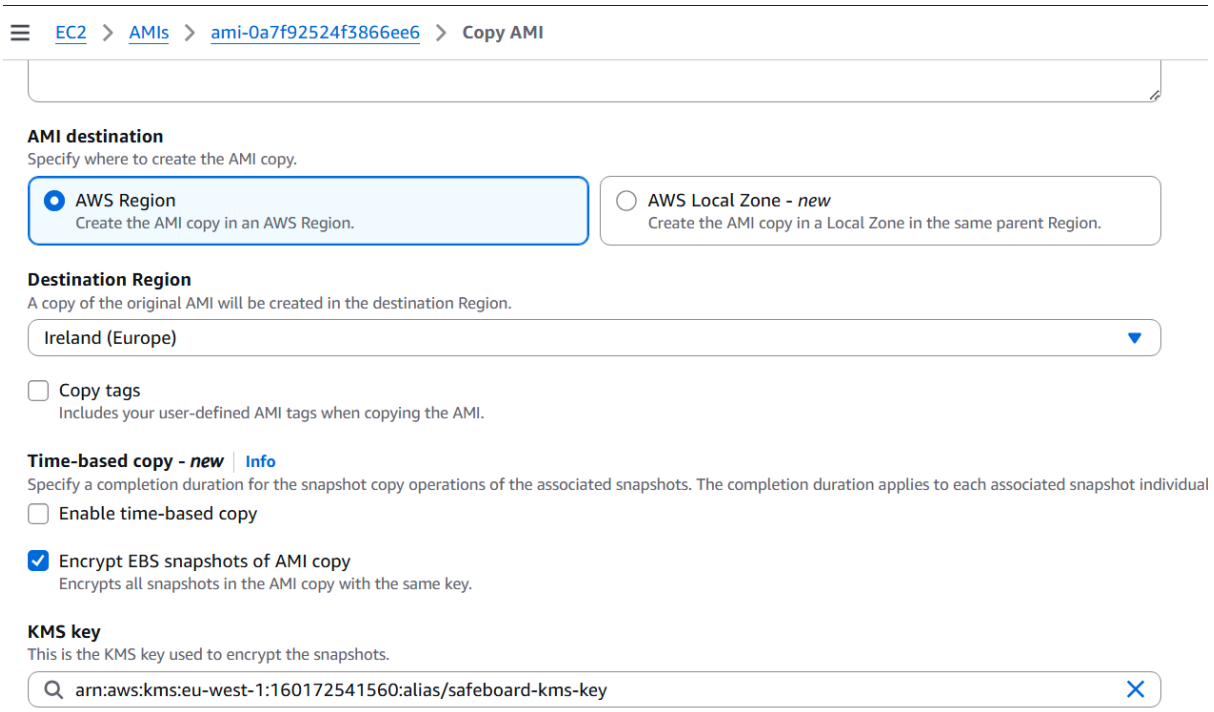
On applique ensuite la clé à notre bucket:



Pour chiffrer EC2 on crée une image AMI:



On chiffre la copie de l'ami avec la clé KMS créée précédemment:

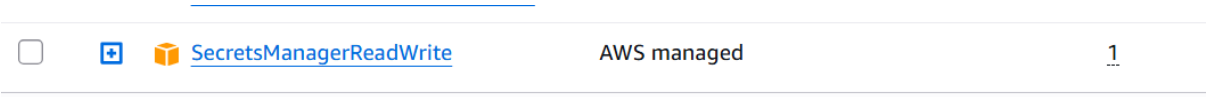


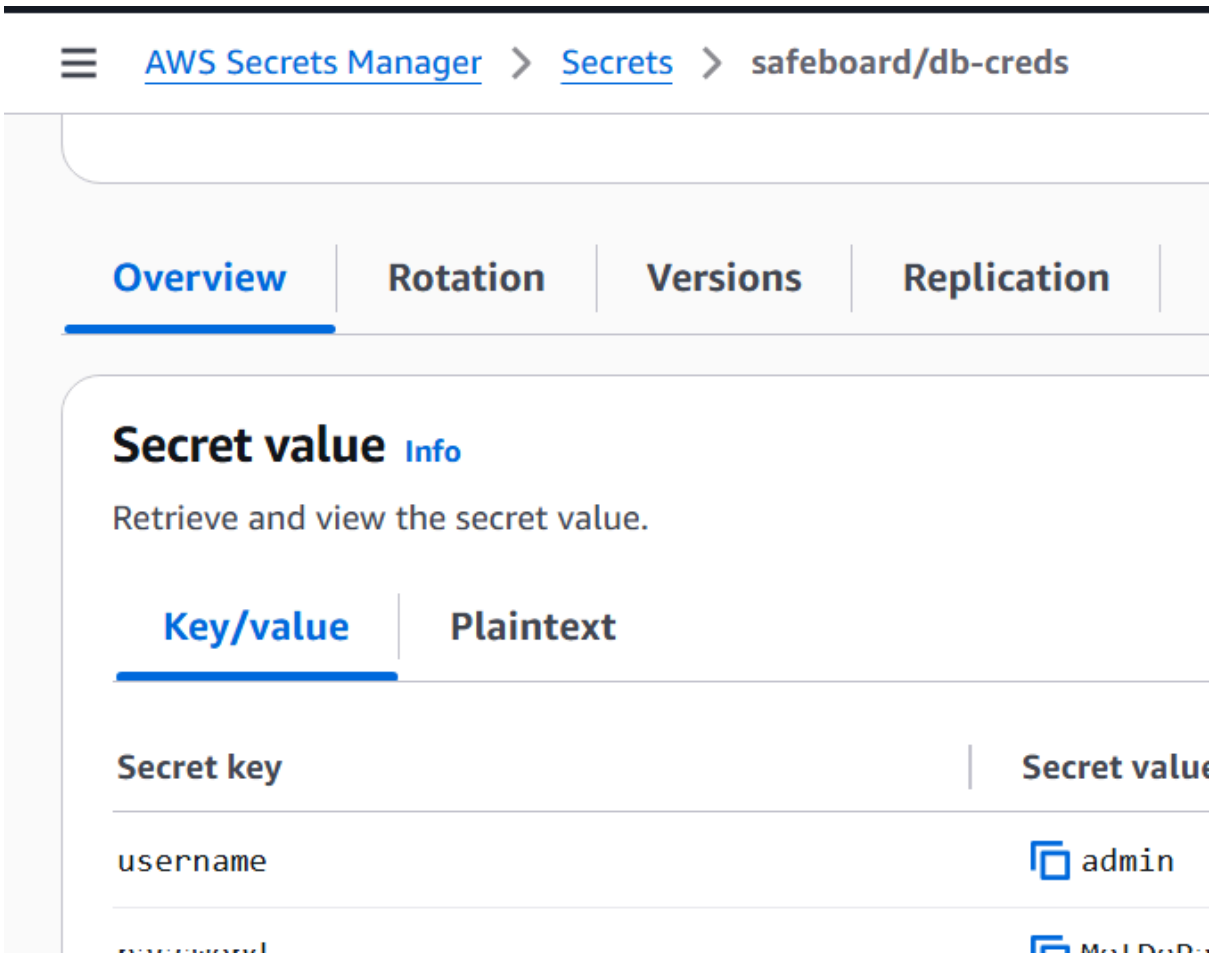
On peut ensuite créer une nouvelle machine ec2 à partir de cet ami, ainsi le volume EBS sera chiffré !

2. Gestion des secrets applicatifs

Intégrer des mots de passe en clair dans le code source est une faille de sécurité majeure. Pour l'éviter, nous avons utilisé **AWS Secrets Manager**. Nous avons créé un secret nommé safeboard/db-creds stockant dynamiquement l'utilisateur et le mot de passe de la base de données RDS.

L'application Python (Flask) utilise la bibliothèque boto3 pour interroger ce service à la volée. Pour que cela fonctionne, la politique IAM SecretsManagerReadWrite a été attachée au rôle de l'instance EC2.





Le code python du serveur web fait ceci avec boto3 pour récupérer les secrets:

```
def get_secret():
    secret_name = "safeboard/db-creds"
    region_name = "eu-west-1"

    # Création du client Secrets Manager
    client = boto3.client('secretsmanager', region_name=region_name)
```

Ensuite on tente une connexion:

```
try:
    connection = pymysql.connect(
        host=secret['host'],          # Récupéré depuis AWS
        user=secret['username'],      # Récupéré depuis AWS
        password=secret['password'], # Récupéré depuis AWS
        database=DB_NAME,
        cursorclass=pymysql.cursors.DictCursor
    )
```

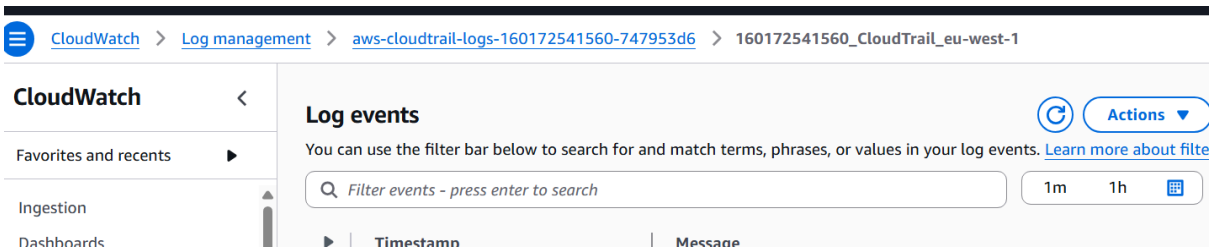
Phase 4 : Surveillance et conformité dans AWS

Objectif : Mettre en place une surveillance continue et assurer la conformité aux meilleures pratiques.

1. Audit et Surveillance Continue

Nous avons activé **AWS CloudTrail** en créant un journal (safeboard-audit-trail). Ce service enregistre chaque appel d'API effectué sur notre compte AWS, offrant une traçabilité totale en cas d'audit ou d'incident de sécurité.

Les journaux CloudTrail sont automatiquement envoyés vers un groupe de logs dans **Amazon CloudWatch**. Pour surveiller activement les tentatives de violation d'accès, nous avons défini un filtre de métrique nommé Filtre-Acces-Refuse. Le modèle de filtre surveille précisément les erreurs UnauthorizedOperation et AccessDenied.

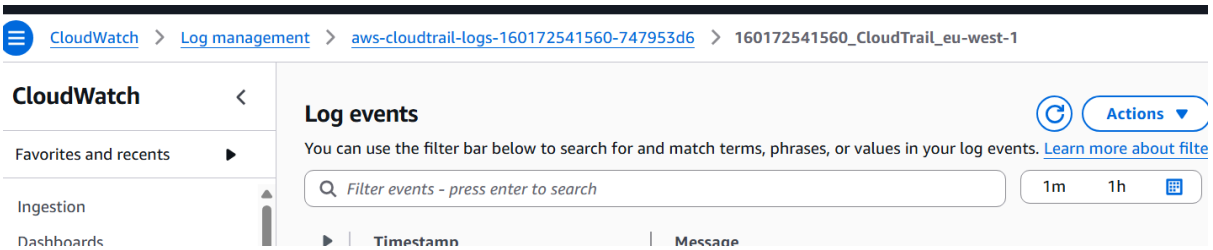


Lorsqu'une telle erreur est détectée, la métrique TentativesNonAutorisees s'incrémente. Une alarme CloudWatch (alarm-metric) est configurée pour s'activer si le seuil dépasse 1.

En cas d'alerte, une notification est envoyée automatiquement par e-mail via une rubrique **Amazon SNS** (Default_CloudWatch_Alarms_Topic).

Trails

Name	Home region	Multi-region trail	ARN	Insights	Organization trail	S3 bucket
safeboard-audit-trail	Europe (Ireland)	Yes	arn:aws:cloudtrail:eu-west-1:160172541560:trail/safeboard-audit-trail	Disabled	No	aws-cloudtrail-logs-160172541560-528a0344



Metric filter "Filtre-Acces-Refuse" has been created.

Metric filters (1)

Find metric filters

Filtre-Acces-Refuse

Filter pattern

{ ($.errorCode = "*UnauthorizedOperation") || ($.errorCode = "AccessDenied*") }

Field selection criteria

-

Metric

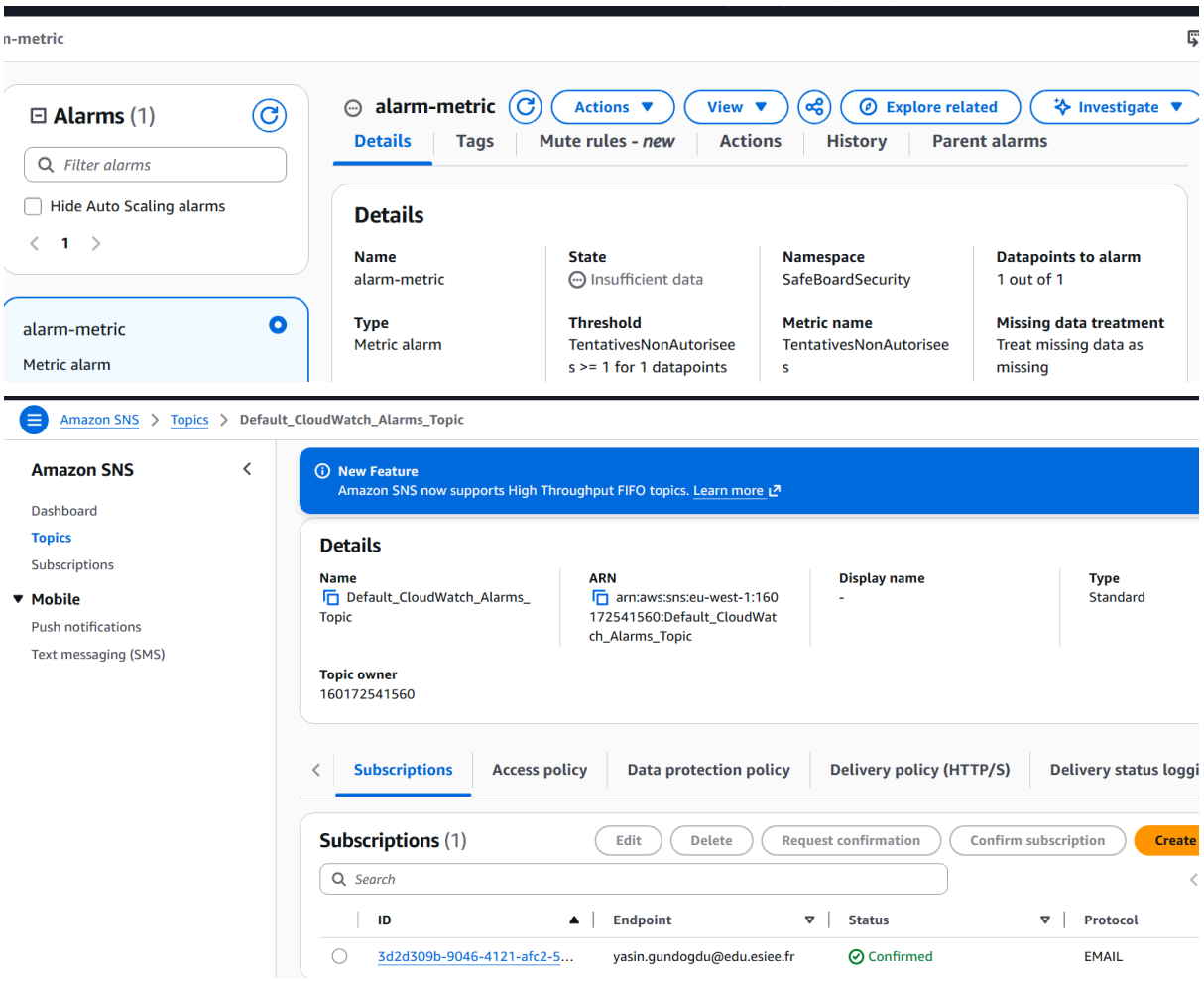
SafeBoardSecurity / TentativesNonAutorisees

Metric value

1

Default value

-



2. Maintien de la Conformité

Pour garantir que notre infrastructure reste sécurisée dans le temps, nous avons déployé **AWS Config**. Ce service évalue en continu nos ressources par rapport à des règles prédéfinies. Nous avons implémenté deux règles critiques :

- **s3-bucket-public-read-prohibited :** Vérifie qu'aucun compartiment S3 ne devient publiquement accessible.
- **encrypted-volumes :** S'assure que tous les disques (volumes EBS) restent chiffrés. Le tableau de bord Config confirme que toutes nos ressources, notamment nos buckets de contenus, affichent un statut Compliant (Conforme).

Delivery channel

Amazon S3 bucket

Create a bucket

Ensure appropriate permissions are available in this S3 bu

S3 Bucket name (required)

config-bucket-160172541560

/AWSLogs/160172541560/Config/eu-west-1

AWS Config > Rules

Rules

A rule is a compliance check that helps you manage your ideal configuration relevant rules and displays the compliance results.

Rules

Filter by compliance status

All

Name
encrypted-volumes
s3-bucket-public-read-prohibited

On peut constater que pour s3, tous nos buckets sont conformes :

Resources in scope

View details | Remediate

All

1

ID	Type	Status	Annotation	Compliance
aws-cloudtrail-logs-...	S3 Bucket	-	-	Compliant
config-bucket-1601...	S3 Bucket	-	-	Compliant
safeboard-content-b...	S3 Bucket	-	-	Compliant
safeboard-logs-bucket	S3 Bucket	-	-	Compliant

Phase 5 : Application de DevSecOps

Objectif : Intégrer la sécurité dans le cycle de vie du développement logiciel.

Pour automatiser la détection des vulnérabilités dans le code source de l'application SafeBoard, nous avons mis en place une pipeline d'intégration et de livraison continues (CI/CD) sécurisée en utilisant **AWS CodePipeline** et **AWS CodeBuild**.

1. Configuration de l'environnement de build

Le code source de l'application (app.py, dossiers templates) a été zippé avec un fichier de configuration essentiel : buildspec.yml, puis uploadé dans un bucket S3 dédié (safeboard-code). Ce fichier buildspec.yml pilote le processus de sécurité (SAST - Static Application Security Testing) :

- Lors de la phase d'installation, un conteneur Linux télécharge **Bandit**, un outil spécialisé dans l'analyse de sécurité du code Python.
- Lors de la phase de *build*, la commande bandit -r . -f txt scanne l'intégralité du code pour y déceler des failles (injections SQL, mots de passe en clair, etc.).

Developer Tools > CodeBuild > Build projects > Create build project

Project type

Select what type of project you would like to create. Info

- Default project
  Create a custom CodeBuild project.

▶ **Additional configuration**

Description, public build access, build badge, concurrent build limit, tags

▼ **Source**

Source 1 - Primary

Source provider

Amazon S3

Bucket

safeboard-code

S3 object key or S3 folder

safeboard-app.zip

Un conteneur linux effectue la tâche:

Provisioning model Info

On-demand
Automatically provision build infrastructure in response to new builds.

Environment image

Managed image
Use an image managed by AWS CodeBuild

Compute

EC2
Optimized for flexibility during action runs

Running mode

Container
Running on Docker container

Operating system

Amazon Linux

Runtime(s)

Standard

Cette option est importante pour qu'il trouve notre fichier builspec avec notre scan:

Buildspec

Build specifications

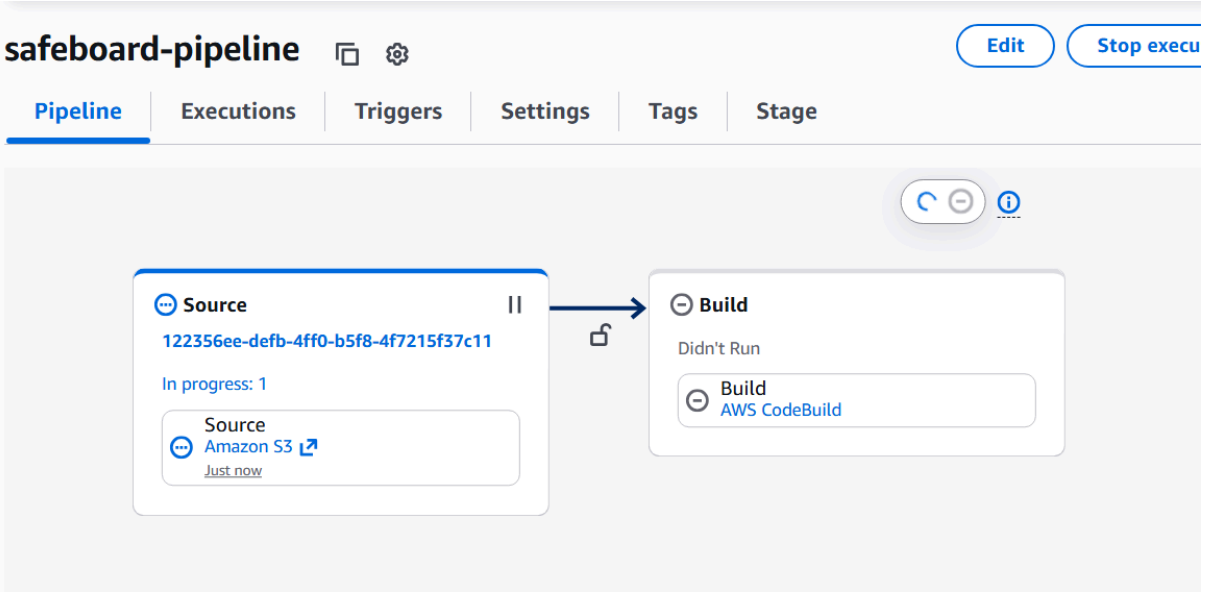
Insert build commands
Store build commands as build project configuration

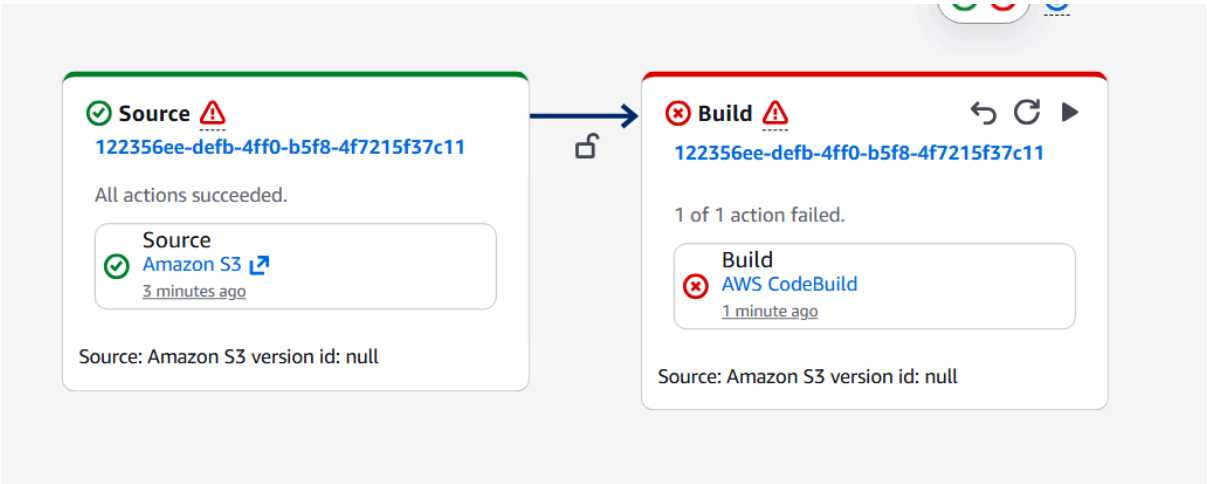
Use a buildspec file
Store build commands in a YAML-formatted buildspec file

2. Test du Pipeline et Résolution (La boucle DevSecOps)

Lors de la première exécution du pipeline, CodeBuild a intentionnellement fait échouer (Failed) le déploiement. L'analyse des journaux de Bandit a révélé deux alertes qui ont stoppé le processus:

- **[B105:hardcoded_password_string] de sévérité faible :** Bandit a identifié la chaîne "safeboard/db-creds" à la ligne 13 et l'a interprétée comme un mot de passe en dur. Il s'agit d'un **faux positif**, car ce n'est que l'identifiant permettant d'interroger AWS Secrets Manager de manière sécurisée.
- **[B104:hardcoded_bind_all_interfaces] de sévérité moyenne :** Bandit a alerté sur l'utilisation de 0.0.0.0 (ligne 85) qui permet à n'importe qui de se connecter. Dans un contexte traditionnel, cela est risqué. Cependant, dans notre architecture réseau en sous-réseau privé, c'est **volontaire et indispensable** pour que l'Application Load Balancer puisse transmettre le trafic web vers l'instance EC2.





Les raisons pour lesquelles le build a fail sont les suivants:

```
Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'safeboard/db-creds'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: ./app.py:13:18
12  def get_secret():
13      secret_name = "safeboard/db-creds"
14      region_name = "eu-west-1"

--------------------------------------------------
>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interfaces.
   Severity: Medium   Confidence: Medium
   CWE: CWE-605 (https://cwe.mitre.org/data/definitions/605.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b104_hardcoded_bind_all_interfaces.html
   Location: ./app.py:85:17
84      # Le serveur écoute sur le port 80 pour que l'Application Load Balancer puisse le joindre
85      app.run(host='0.0.0.0', port=80)
```

Conformément aux pratiques DevSecOps, la résolution ne consiste pas à désactiver l'outil de scan, mais à traiter l'alerte. Nous avons ajouté des commentaires `# nosec` dans le code source Python à la fin des lignes concernées. Ces annotations informent l'outil d'analyse que ces lignes ont été humainement auditées et qu'elles sont jugées sécurisées dans le contexte de notre infrastructure.

(Suite à cette modification, le fichier zip a été mis à jour dans S3, déclenchant automatiquement une nouvelle exécution de CodePipeline qui s'est cette fois terminée avec succès, validant ainsi la robustesse de notre cycle de développement sécurisé).

Recherche Théorique : Les attaques adverses ciblant les plateformes de contenus numériques

Objectif : Répondre à la consigne de recherche sur l'état de l'art du cahier des charges

Les plateformes de gestion de contenus numériques comme SafeBoard centralisent des actifs de grande valeur (documents d'identité, contrats, imageries). Elles sont donc des cibles privilégiées pour diverses attaques adverses visant à compromettre la confidentialité, l'intégrité ou la disponibilité des données.

Types d'attaques adverses identifiées

1. **L'injection de contenu malveillant (Malicious Uploads) :** Un attaquant contourne les vérifications d'upload pour téléverser un fichier vérolé (ex. un script exécutable camouflé avec une extension `.pdf` ou `.jpg`). Une fois stocké, ce fichier peut être exécuté par le serveur ou infecter l'ordinateur d'un administrateur RH qui tenterait de l'ouvrir.
2. **Attaques de type Insecure Direct Object References (IDOR) :** Un utilisateur légitime modifie les paramètres de l'URL (comme l'ID d'un document) pour accéder aux pièces justificatives d'un autre employé, violant ainsi la confidentialité.
3. **Déni de Service (DoS/DDoS) sur le stockage :** En l'absence de limites, un attaquant automatise l'upload massif de fichiers très lourds, saturant la bande passante, épuisant l'espace de stockage alloué, et augmentant drastiquement la facture cloud de l'entreprise (Denial of Wallet).
4. **Empoisonnement de données (Data Poisoning) :** Plus spécifique aux contenus numériques utilisés pour entraîner des modèles d'IA, l'attaquant modifie subtilement les pixels d'une image ou les métadonnées d'un document. Pour l'œil humain, le document semble intact, mais il perturbera les algorithmes de classification (par exemple, un système automatisé de vérification de pièces d'identité rejettera les vrais documents).

Stratégies de défense et bonnes pratiques implémentées

Pour contrer ces menaces, notre architecture SafeBoard a intégré plusieurs paradigmes de défense :

- **Contre les Uploads Malveillants :** Utilisation de processus DevSecOps (SAST) pour s'assurer que le code backend assainit les noms de fichiers (`secure_filename`) avant le transfert vers S3. Sur AWS, l'idéal est de brancher une fonction Lambda sur le bucket S3 qui scanne le fichier avec un anti-virus (via API) dès son dépôt, avant de le déplacer vers un bucket final "propre".
- **Contre les failles d'accès (IDOR) :** La mise en place de politiques strictes sur S3 ("Block Public Access") combinée à l'utilisation d'URL pré-signées éphémères

générées par le backend garantit que seul un utilisateur authentifié peut télécharger un fichier précis.

- **Contre le déni de dervice :** L'Application Load Balancer (ALB) absorbe une partie du trafic anormal. Dans un environnement de production, l'ajout d'AWS WAF (Web Application Firewall) permettrait d'imposer un "Rate Limiting" pour empêcher l'envoi massif de requêtes.

Conclusion

Le projet SafeBoard démontre avec succès la création d'une architecture Cloud sécurisée de bout en bout. En associant une ségrégation réseau stricte (VPC, sous-réseaux privés, NACL) , un chiffrement systématique des données au repos (AWS KMS, S3, EBS) et des accès audités (Secrets Manager, CloudTrail, Config) , la plateforme offre des garanties solides pour la gestion de contenus sensibles.

Enfin, l'automatisation des analyses de sécurité via notre pipeline CodePipeline/CodeBuild illustre la maturité de la philosophie DevSecOps appliquée au projet : la sécurité n'est pas une surcouche finale, mais une exigence intégrée dès l'écriture du code.